

Parkta Kavşak

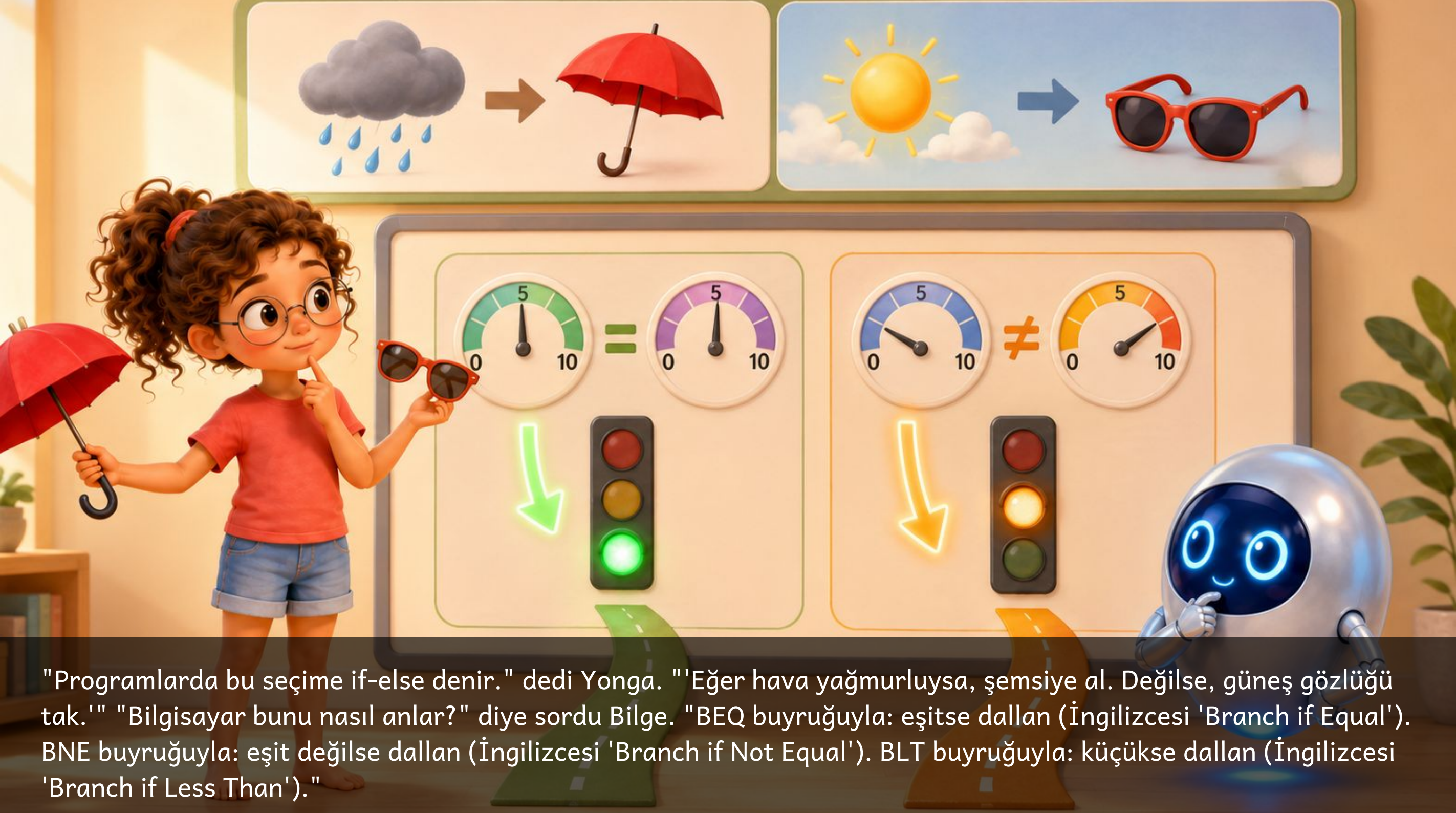
Bilge ve Yonga'nın Maceraları — Buyrukların Dünyası



Prof. Dr. Oğuz Ergin



Bilge ve Yonga parkta yürürken bir kavşağa geldiler. "Sola dönersek göl, sağa dönersek kafe." dedi Bilge. "Hangisine gitsek?" Yonga güldü: "Bu tam bir dallanma işlemi! Bilgisayarlar da her an böyle seçimler yapar. 'Koşul doğruysa sol yola git, yanlışsa sağa.'"



"Programlarda bu seçime if-else denir." dedi Yonga. "'Eğer hava yağmurluysa, şemsiye al. Değilse, güneş gözlüğü tak.'" "Bilgisayar bunu nasıl anlar?" diye sordu Bilge. "BEQ buyruğuyla: eşitse dallan (İngilizcesi 'Branch if Equal'). BNE buyruğuyla: eşit değilse dallan (İngilizcesi 'Branch if Not Equal'). BLT buyruğuyla: küçükse dallan (İngilizcesi 'Branch if Less Than')."



"BEQ x5, x6, 20." dedi Yonga. "Bu şu anlama gelir: Eğer x5 ve x6 eşitse, Program Sayacını 20 adres ileri götür. Eşit değilse, bir sonraki buyruğa geç, hiç atlamadan devam et." "Demek atlama olmadığında özel bir şey yapılmıyor." dedi Bilge. "Evet! Zaten bir sonraki buyruğa devam etmek varsayılandır."



"Döngüler ne?" diye sordu Bilge. "Döngü, bir şeyi belirli kez ya da bir koşul bozulana kadar tekrarlamak demek." dedi Yonga. "'10 kez zıpla' demek gibi. Nasıl mı? Dallanma buyruğuyla! Her turda sayacı bir artır. Sayaç 10'a ulaşınca koşul bozulur ve döngü durur." "Döngü, geriye doğru bir atlama!" dedi Bilge.



"Koşulsuz atlama da var." dedi Yonga. "JAL: 'Jump And Link', atla ve bağlantıyı kaydet. 'Şu adrese git' der, koşul yok, doğrudan gider. Ama 'bağlantı kaydet' kısmı çok önemli: Geri dönmek için nereden geldiğini hatırlıyor!" "Hansel ile Gretel gibi." dedi Bilge. "Orman yoluna taşlar bıraktılar!" "Aynen öyle! JAL de dönüş adresini ra (x1) yazmacına bırakır."



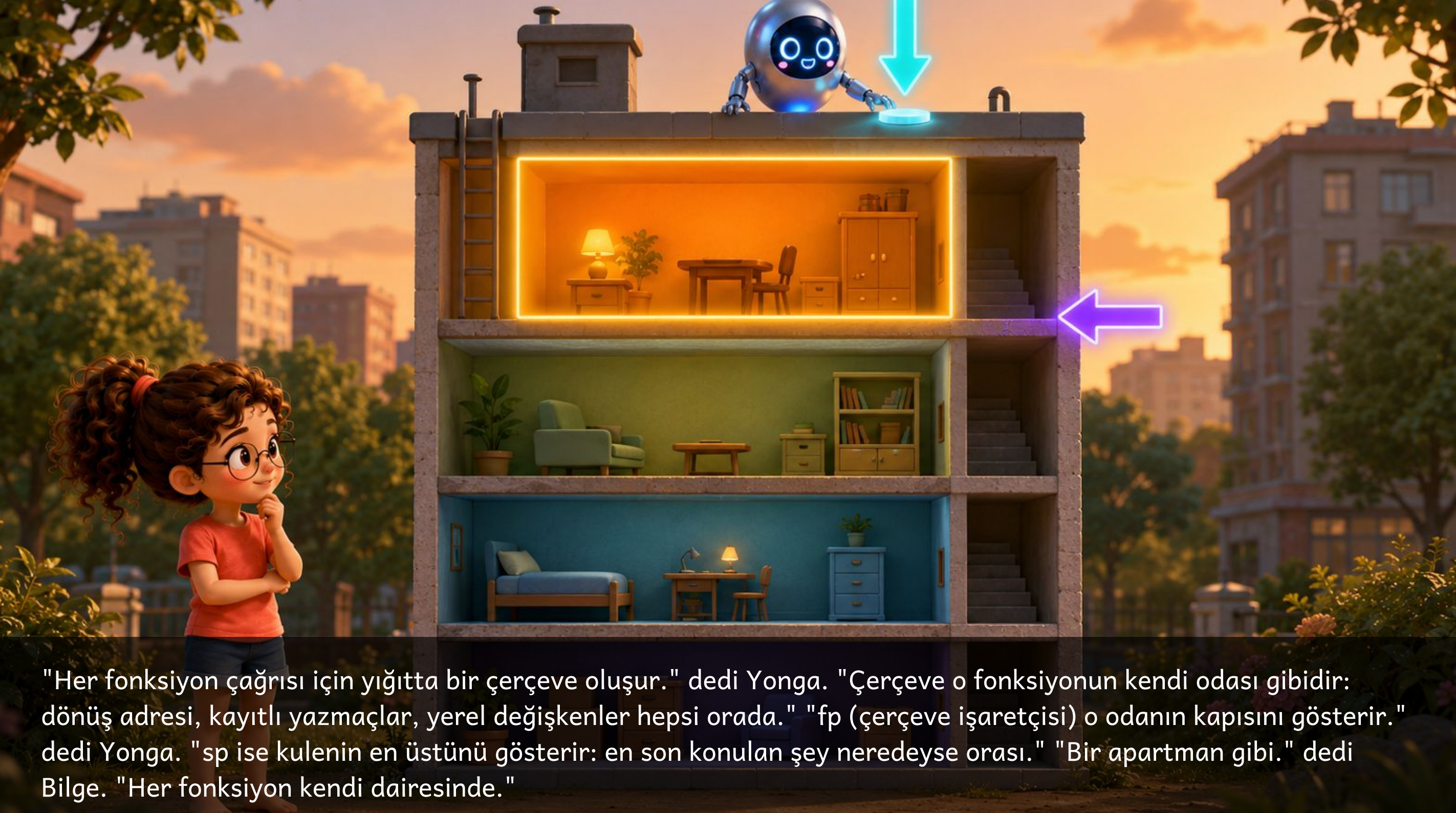
"Programlarda aynı işi tekrar tekrar yazmak yerine, o işi bir kez yazarsın ve defalarca çağırırsın. Buna altyordam ya da fonksiyon denir." dedi Yonga. "Tarif defteri gibi!" dedi Bilge. "'Pasta yap' tarifini bir kez yazarsın, sonra 'pasta yap' dersin: tarife bakarsın, yaparsın, geri dönersin." "Harika benzetme! Çağırma → gitme → yapma → geri dönme."



"Bir fonksiyon çağırıyorum." dedi Bilge. "Fonksiyon başka bir fonksiyon çağırıyor. O da başka bir fonksiyon çağırıyor. Sonunda nasıl geri döneceğimi bilebilir miyim?" "İşte yığıt tam burada devreye giriyor!" dedi Yonga. "Her fonksiyon çağırısında dönüş adresi yığıtta konur. Geri dönerken yığıttan alınır. Kim son koyduysa birinci çıkar!"



"Fonksiyon çağrılırken şunlar yığta konur." dedi Yonga: "1. Dönüş adresi (nereden gelindi) 2. Kullanılan yazmaçların değerleri (ezilmesin diye) 3. Fonksiyonun yerel değişkenleri" "Ve geri dönerken?" diye sordu Bilge. "Tam tersi sırayla yığıttan çıkarılır: yazmaçlar eski değerlerine döner, yürütme kaldığı yerden devam eder."

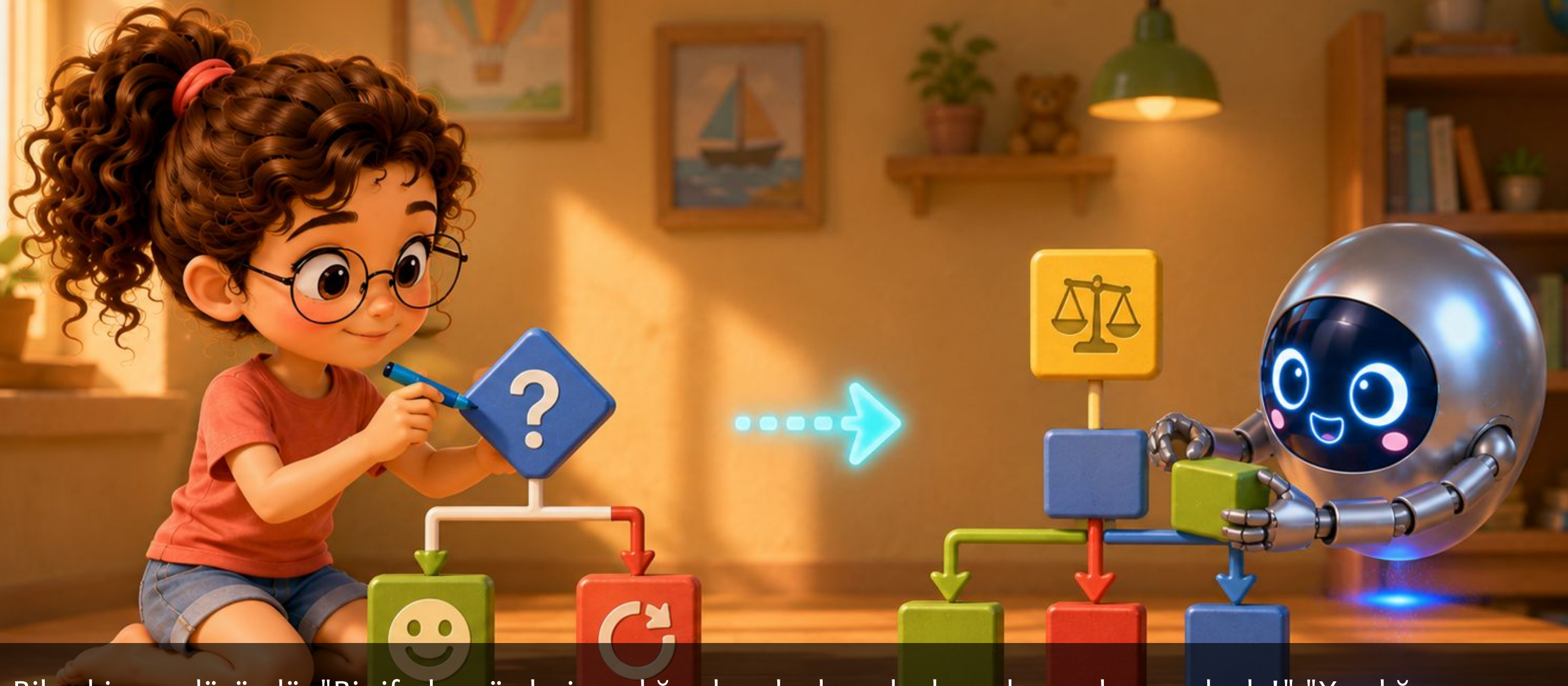


"Her fonksiyon çağrısı için yığıtta bir çerçeve oluşur." dedi Yonga. "Çerçeve o fonksiyonun kendi odası gibidir: dönüş adresi, kayıtlı yazmaçlar, yerel değişkenler hepsi orada." "fp (çerçeve işaretçisi) o odanın kapısını gösterir." dedi Yonga. "sp ise kulenin en üstünü gösterir: en son konulan şey neredeyse orası." "Bir apartman gibi." dedi Bilge. "Her fonksiyon kendi dairesinde."



"JAL'ın bir kardeşi var: JALR." dedi Yonga. "JAL doğrudan bir adrese atlar. JALR ise bir yazmaçtaki adrese atlar. Bu çok güçlüdür! Çünkü hangi fonksiyonu çağıracağını önceden bilmesen de olur: sadece adresi yazmaca koyarsın, JALR oraya gider." "Telefon rehberi gibi." dedi Bilge. "Numarayı önceden bilmiyorsun, rehber bakıyorsun, arıyorsun."





Bilge bir şey düşündü: "Bir if-else cümlesi yazdığımda aslında ne kadar çok şey oluyor arkada!" "Yazdığımız programı işlemcinin diline çeviren bir araç var: derleyici." dedi Yonga. "Derleyici senin if-else'ini karşılaştırma buyruğuna ve dallanma buyruğuna çevirir. Koşul tutmuyorsa BEQ ya da BNE ile öteki dala atlar. Tüm mantık sıfır ve birlerle!"



"Bir for döngüsü yazdığımda da aynı mı?" diye sordu Bilge. "Evet! Derleyici şuna çevirir: 1. Sayacı sıfırla: $x5 = 0$ 2. Döngü başı: $x5 < 10$ mı? Değilse döngü sonundaki buyruğa atla. 3. Döngü gövdesini yürüt. 4. Sayacı artır: $x5 = x5 + 1$ 5. Döngü başına geri atla (JAL ya da geriye dallanma)." "Vay be! Her döngü aslında bir geri atlama." dedi Bilge.



O akşam Bilge'nin annesi ona "Odanı topla" dediğinde gülümsedi. "Bu bir fonksiyon çağrısı." diye düşündü. "Ana program (annem) alt programa (ben) çağrı yaptı. Ben işimi bitirince geri dönüp rapor vereceğim." "Bitti!" diyerek mutfağa koştu. Annesinin yüzündeki gülümseme bir dönüş değeri gibiydi.

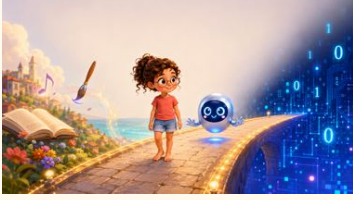


Bilge yatarken düşündü: "Dallan, atla, geri dön. Koşul doğruysa bir yol, yanlışsa başka yol. Tekrar et, dur, devam et. Belki de ben de her gün böyle çalışıyorum: kararlar veriyorum, görevleri tamamliyorum, sonuçları bildiriyorum. Bilgisayar benden o kadar da farklı değil."

Bugün Ne Öğrendik?

- Dallanma: Bir koşul doğruysa (ya da yanlışsa) farklı bir buyruğa atlamak. BEQ, BNE, BLT gibi buyruklar kullanılır.
- Döngü: Bir işlemi belirli kez ya da koşul bozulana kadar tekrarlamak, geriye doğru dallanmayla gerçekleşir.
- JAL: Koşulsuz atlama. Dönüş adresini ra yazmacına kaydeder.
- JALR: Yazmaçtaki adrese atlama. Hangi fonksiyona gidileceği önceden bilinmese de olur.
- Altyordam (fonksiyon): Defalarca çağrılabilen, iş bitince geri dönen kod bloğu.
- Yığıt çerçevesi: Her fonksiyon çağrısı için yığıtta ayrılan alan: dönüş adresi, kayıtlı yazmaçlar, yerel değişkenler burada.
- sp (yığıt işaretçisi): Yığıtın en üstünü gösterir.
- ra (dönüş adresi yazmacı): Fonksiyonun geri döneceği adresi saklar.

Buyrukların Dünyası



Kitap 3.1: İki Dünyanın Köprüsü

Buyruk kümesi mimarisi ve Getir-Çöz-Yürüt



Kitap 3.2: Sıranın Üstündeki Kalemler

Yazmaçlar, bellek düzeni ve bayt sırası



Kitap 3.3: Zarfın Üstündeki Bölmeler

Buyruk biçimleri ve adresleme kipleri



>> Kitap 3.4: Parkta Kavşak

Dallanma, döngüler ve altyordamlar



Kitap 3.5: Mektup Fabrikası

Derleme zinciri: derleyici, çevirici, bağlayıcı



Kitap 3.7: Sihirli Maskeler

Mantık buyrukları: VE, VEYA, DEĞİL



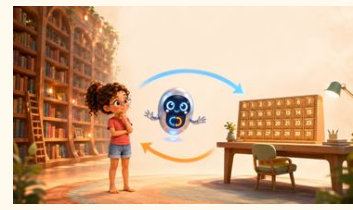
Kitap 3.8: Sıfırın Gücü

x0 yazmacı ve sıfırın gücü



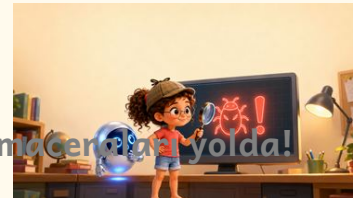
Kitap 3.9: Kulelerin Oyunu

Yığıt ve altyordamlar



Kitap 3.10: Taşıyıcılar

Veri aktarma buyrukları: yükle ve sakla



Kitap 3.11: Dedektif Bilge ve Kayıp Sonuç

Hata ayıklama: ara nokta, adım adım yürütme

Bilge ve Yonga'nın yeni macerası yolda!

Bilge ve Yonga'nın Tüm Maceraları

Her kitap tek bir fikri, baştan sona bir hikâyeyle anlatır

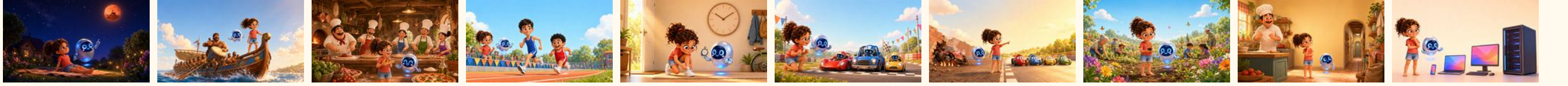
Kumdan Bilgisayara

12 kitap



Hız ve Güç

10 kitap



Buyrukların Dünyası

11 kitap



Bütün kitaplar ücretsiz, çevrimiçi:
bilgeveyonga.oguzergin.net

KÜNYE

© 2026 Prof. Dr. Oğuz Ergin

Parkta Kavşak

Bilge ve Yonga: Bilgisayar Mimarisi Çocuk Kitapları Serisi

Ankara, 2026

Sürüm 1.0, 5 Ağustos 2026

Bu eser, Creative Commons Atıf-GayriTicari-Türetilemez 4.0 Uluslararası

(CC BY-NC-ND 4.0) lisansı ile açık erişime sunulmuştur.

Lisans metni: <https://creativecommons.org/licenses/by-nc-nd/4.0/deed.tr>

LİSANSIN KAPSAMADIĞI TÜM HAKLAR SAKLIDIR.

“Bilge ve Yonga” seri adı, “Bilge” ve “Yonga” karakter adları ile figürleri, seri amblemi, marka hakları, eser sahibinin adı ve unvanı ile manevi haklar bu lisansın kapsamı dışındadır ve ayrı yazılı izne bağlıdır.

Metin ve veri madenciliği ile yapay zekâ modeli eğitimi bakımından haklar açıkça saklı tutulmuştur.

Eserler olduğu gibi sunulmaktadır; belirli bir amaca uygunluk konusunda garanti verilmez.

Ticari kullanım izni ve sorularınız için: bilgi@oguzergin.net

Telif ve lisans politikası: bilgeveyonga.oguzergin.net/telif-ve-lisans.html

Atıf: Ergin, O. (2026). Bilge ve Yonga: Bilgisayar Mimarisi Çocuk Kitapları Serisi. <https://bilgeveyonga.oguzergin.net>

Bilge ve Yonga © 2026 Prof. Dr. Oğuz Ergin · CC BY-NC-ND 4.0

Bilge ve Yonga'nın Maceraları

Buyrukların Dünyası

Prof. Dr. Oğuz Ergin

© 2026 · CC BY-NC-ND 4.0